Vehicle Parking App – V2

Final Report for the Modern Application Development-2 Project

1. Student Details

- **Name:** Anish Kumar Pal
- **Roll Number:** 22f1001091
- **Course:** Modern Application Development Project - II
- **Email:** 22f1001091@ds.study.iitm.ac.in

---

2. Project Details

Project Title:

Vehicle Parking Management System

Problem Statement:

Design and implement a comprehensive parking management system that automates vehicle entry/exit, slot allocation, billing, and provides real-time notifications to users while maintaining security and user experience.

Approach:

- Analyzed requirements for both user and admin roles.
- Designed the database schema to efficiently manage lots, spots, users, and reservations.
- Implemented the backend using Flask (Python) with RESTful API endpoints for AJAX and integration.
- Developed user and admin interfaces using HTML, CSS, and JavaScript (with Bootstrap for styling).
- Ensured secure authentication and role-based access control.
- Documented the API using OpenAPI (YAML format).
- Integrated **Flask-APScheduler** to handle scheduled background jobs, including daily reminders and monthly reports.

- Implemented a **notification system** with multiple channels (Email, Google Chat, SMS) to improve user engagement and reliability.
- Added **payment functionality** supporting multiple methods (Card, UPI), with transaction tracking and default payment method support.
- Built **reporting features** such as CSV export, PDF generation (via reportlab), and monthly usage summaries for both users and admins.
- Applied **database optimizations** include indexing and cascading rules for performance and integrity.
- Followed a **modular code architecture** (models, routes, services, utilities) to improve maintainability and scalability.
- Added **real-time occupancy tracking dashboards** and user statistics for insights into usage patterns
- Adopted **security best practices** such as input validation, password hashing, CSRF protection, and SQL injection prevention.
- Deployed using **gunicorn** and **Werkzeug**, with timezone support handled by **pytz** (IST/UTC+5:30).
- Enabled **Progressive Web App (PWA) support**, allowing offline access and mobile usability.

---

3. Frameworks and Libraries Used

- Flask – Lightweight Python web application framework
- Flask-**Login** – User authentication and session management
- Flask-**SQLAlchemy** – Object Relational Mapper (ORM) for database interaction
- Flask-**APScheduler** – Background job scheduling (reminders, reports)
- Bootstrap – Responsive frontend styling framework
- Jinja**2** – Templating engine for dynamic HTML generation
- SQLite – Lightweight relational database for storage
- SMTP **(Gmail)** – Email notifications, with **requests** library for Google Chat API calls
- reportlab – PDF report generation

- Werkzeug**, gunicorn, pytz** – Deployment utilities and timezone handling

- Other – Python standard libraries along with JavaScript, HTML5, and CSS3 for frontend interactivity and design

4. ER Diagram (Database Design)

Tables:

- **User:**
  id, username, email, password_hash, is_admin, full_name, address, pin_code, phone, vehicle_no, created_at, created_at, notification_preference, gchat_email, reminder_time, report_format, last_reminder_date
- **ParkingLot:**
  id, name, prime_location_name, price, address, pin_code, maximum_number_of_spots, created_at
- **ParkingSpot:**
  id, lot_id (FK), spot_number, status, created_at
- **ParkingReservation:**
  id, user_id (FK), spot_id (FK), vehicle_no, parking_timestamp, leaving_timestamp, parking_cost, active, payment_status, payment_date, payment_method, transaction_id
- **SavedPaymentMethod:**
  id, user_id (FK), payment_type, card_last_four, card_type, card_holder_name, card_expiry, upi_id, is_default, created_at, last_used

Relationships:

- One ParkingLot has many ParkingSpots
- One ParkingSpot belongs to one ParkingLot
- One User can have many ParkingReservations
- One ParkingReservation is linked to one User and one ParkingSpot
- One User is linked to Many SavedPaymentMethods
- One SavedPaymentMethod belongs to one User

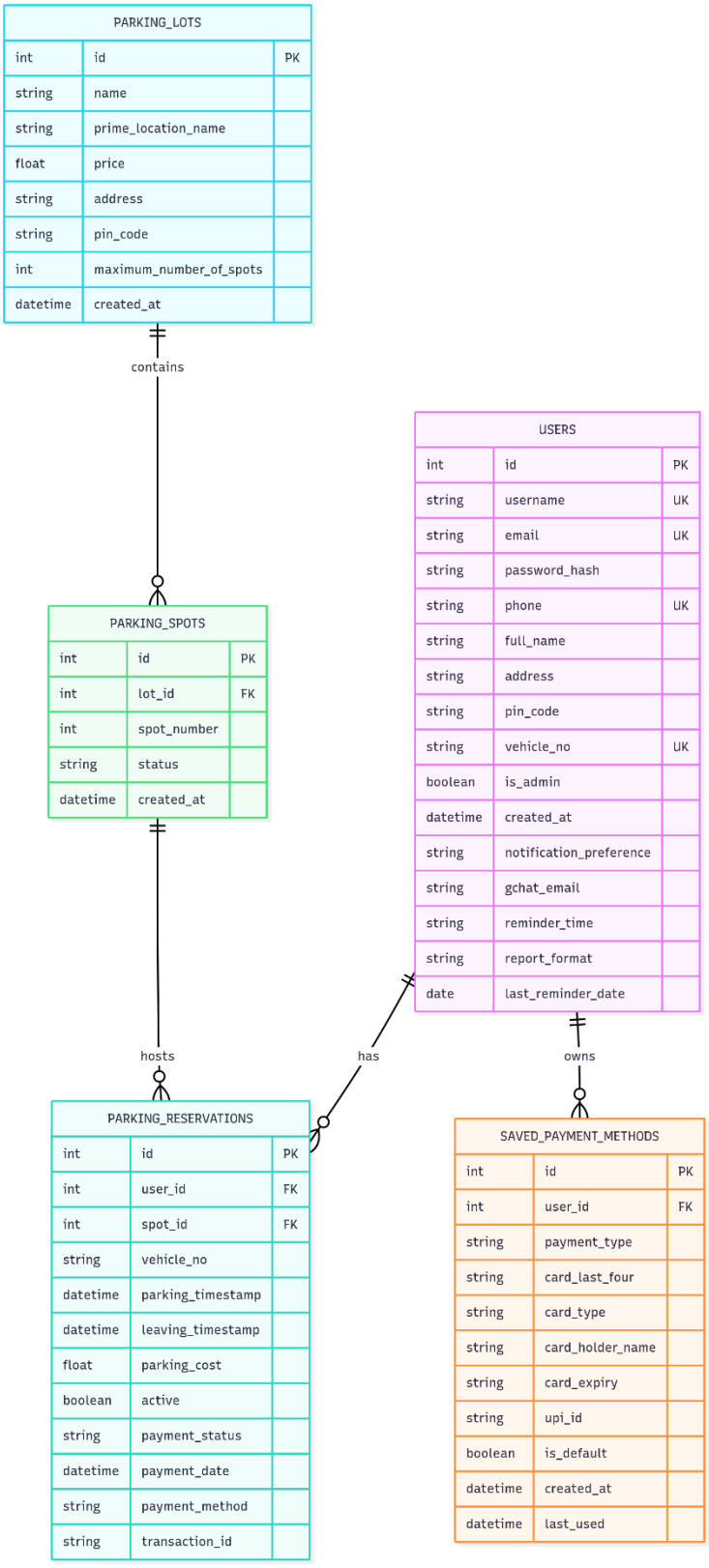

Fig. 1 Entity Relationship Diagram

5. API Resource Endpoints

- GET /api/available-spots/{lot_id} – Get available parking spots for a specific lot
- GET /api/user-stats – Get statistics for the current user
- GET /api/parking-stats – Get parking lot statistics (admin only)
- GET /api/user-stats - Personal user analytics
- Additional endpoints include:
    - Booking a spot
    - Releasing a reservation
    - User account management
    - Admin management operations
      (Detailed definitions in api_definition.yaml)

---

6. Presentation Video

Link: Video

This report has been submitted as part of the project requirements. All code and documentation are included in the project submission.